

Krytyczne funkcjonalności sieciowe



Wykrywanie opóźnień w komunikacji oraz sposoby radzenia sobie z nimi

```
const bool brakNowegoSterowania = !czySiecJednostronna() && !m_maNoweSterowanie;  
if (brakNowegoSterowania) {  
    ++m_brakiSterowaniaPodRzad;  
    lblSterowanieSieci->setText(QString("Sterowanie: brak nowego u, trzymam poprzednie (%1 krokow).")  
                                .arg(m_brakiSterowaniaPodRzad));  
}
```

Gdy pakiet nie dotrze na czas, aplikacja podtrzymuje ciągłość pracy realizując obliczenia na podstawie ostatniej odebranej wartości, wysyłając jednocześnie flagę do regulatora o braku nowych danych.

Funkcja weryfikuje, czy wyliczone opóźnienie sieci i zgubione kroki mieszczą się w dopuszczalnych limitach. Jeśli sieć nie nadąża, program automatycznie zapala czerwoną lampkę, dając operatorowi natychmiastowy, wizualny sygnał o problemach z płynnością transmisji.

```
qint64 czasNadania = probka["nadano_ms"].toString().toLongLong();  
qint64 aktualnyCzas = QDateTime::currentMSecsSinceEpoch();  
qint64 opoznienieSieci = 0;  
  
if (czasNadania > 0) {  
    opoznienieSieci = aktualnyCzas - czasNadania;  
}  
int roznicaKrokow = m_numerProbkiSieciowej - seq;  
ustawStatusSynchronizacji(roznicaKrokow, opoznienieSieci);
```

Protokół komunikacyjny

Tryby pracy { Nieokreslony, Regulator, Obiekt };

Regulator uruchamia serwer TCP i czeka na połączenie.

Obiekt łączy się do regulatora jako klient.

Typy pakietów: PAKIET_KONFIG, PAKIET_PROBKA

```
void Sieci::set_tryb(Tryb tryb)
{
    czyscPolaczenie();
    m_tryb = tryb;

    if (m_tryb == Regulator) {
        if (!m_server.listen(QHostAddress::Any, m_port)) {
            qDebug() << "Sieci: nie można nasłuchiwać:" << m_server.errorString();
            emit statusZmieniony(QString("Błąd: %1").arg(m_server.errorString()));
            m_tryb = Nieokreslony;
            return;
        }
        qDebug() << "Sieci: tryb REGULATOR, nasłuchiwanie na porcie" << m_port;
        emit statusZmieniony(QString("Regulator: nasłuchiwanie na porcie %1...").arg(m_port));
    }
    else if (m_tryb == Obiekt) {
        m_socket = new QTcpSocket(this);
        podlaczSocket();
        qDebug() << "Sieci: tryb OBIEKT, łączenie z" << m_host << ":" << m_port;
        emit statusZmieniony(QString("Obiekt: łączenie z %1:%2...").arg(m_host).arg(m_port));
        m_socket->connectToHost(m_host, m_port);
    }
    else {
        emit statusZmieniony("Rozłączony");
    }
}
```

```
enum Tryb { Nieokreslony, Regulator, Obiekt };

enum TypPakietu : quint8 {
    PAKIET_KONFIG = 1,
    PAKIET_PROBKA = 2
};

enum SerializacjaProbek { SER_BINARNA, SER TEKSTOWA };
```

Ramka protokołu



```
void Sieci::wyslijPakiet(TypPakietu typ, const QByteArray &payload)
{
    if (!czyPołączony())
        return;

    QByteArray naglowek;
    QDataStream out(&naglowek, QIODevice::WriteOnly);

    out.setVersion(QDataStream::Qt_5_15);
    out << static_cast<quint8>(typ);
    out << static_cast<quint32>(payload.size());

    m_socket->write(naglowek);
    m_socket->write(payload);
    m_socket->flush();
}
```

Sposób serializacji danych

```
QByteArray serializujProbkeBinarnie(const QJsonObject &probka)
{
    QByteArray payload;
    QDataStream out(&payload, QIODevice::WriteOnly);
    out.setVersion(QDataStream::Qt_5_15);

    out << FORMAT_PROBKI;
    out << probka.value("rodzaj").toString();
    out << probka.value("komenda").toString();
    out << static_cast<qint32>(pobierzInt(probka, "seq"));
    out << pobierzDouble(probka, "t");
    out << pobierzDouble(probka, "dt");
    out << pobierzDouble(probka, "w");
    out << pobierzDouble(probka, "y");
    out << pobierzDouble(probka, "y_poprzednie");
    out << pobierzDouble(probka, "e");
    out << pobierzDouble(probka, "u");
    out << pobierzDouble(probka, "P");
    out << pobierzDouble(probka, "I");
    out << pobierzDouble(probka, "D");
    out << pobierzDouble(probka, "shadow_y");
    out << pobierzDouble(probka, "shadow_u");
    out << static_cast<qint32>(pobierzInt(probka, "taktowanie"));
    out << pobierzInt64(probka, "nadano_ms");
    out << probka.value("brak_nowego_u").toBool();

    return payload;
}
```

```
QJsonObject MainWindow::zbudujKonfiguracjeJson() const
{
    QJsonObject root;

    QJsonObject jArx;
    QJsonArray arrA, arrB;
    for (double v : arx.getA())
        arrA.append(v);
    for (double v : arx.getB())
        arrB.append(v);
    jArx["A"] = arrA;
    jArx["B"] = arrB;
    jArx["k"] = arx.getOpoznienie();
    jArx["szum"] = arx.getSzum();
    root["ARX"] = jArx;

    QJsonObject jPid;
    jPid["Kp"] = pid.getKp();
    jPid["Ti"] = pid.getTi();
    jPid["Td"] = pid.getTd();
    jPid["Metoda"] = (int) pid.getMetodaCalkowania();
    root["PID"] = jPid;

    QJsonObject jGen;
    jGen["Typ"] = (int) gen.getTyp();
    jGen["Pocz"] = gen.getPoczatkowa();
    jGen["Zm"] = gen.getZmiana();
    jGen["Czas"] = gen.getCzasZmiany();
    jGen["Wyp"] = gen.getWypelnienie();
    jGen["Akt"] = gen.getCzasAktywacji();
    root["Gen"] = jGen;

    root["Interwal"] = petla.getInterwal();
    root["OknoCzasowe"] = oknoCzasowe;
    root["TaktowanieSieci"] = comboTaktowanieSieci->currentIndex();

    return root;
}
```